Wordlists

Both your John the Ripper and Hashcat guides landed on the same conclusion from different directions: cracking speed is a function of *(hash algorithm cost) x (candidate quality)* — and this guide is entirely about that second variable, the one neither tool actually controls. A wordlist is the fuel, not the engine; the fastest GPU in the world running against a bad wordlist still fails. Worth locking in the distinction immediately: **Part 1 covers curated/existing wordlists** — the pre-built corpora you reach for first, and how to evaluate/choose between them — **Part 2 covers generating custom wordlists** — building a targeted list from scratch when a generic corpus won't cut it. This connects directly back to both cracking guides' repeated point that wordlist relevance, not tool choice, is usually the actual bottleneck.

---

1. Why Wordlist Quality Dominates Everything Else — The Foundation

A wordlist's value isn't its size — it's how well it models the **actual population of passwords a real human, or a real organization's naming conventions, would plausibly produce**. This is a statistical targeting problem, not a coverage problem:

```
A 14-million-entry generic wordlist tried against a SPECIFIC organization's employees
will often crack FEWER passwords than a 500-entry list built specifically from that or
ganization's naming conventions, breach history, and observed patterns.

Generic wordlist  → broad coverage of "how humans in general choose passwords" - high
value as a FIRST pass

Targeted wordlist   → narrow coverage of "how THIS population specifically chooses pas
swords" - often dramatically higher hit rate per candidate tried
```

The single fact underlying every section below — every wordlist, however it was built, is really just an approximation of a target population's password-choice behavior, and the closer that approximation matches reality, the fewer candidates you need to try to succeed. This is exactly why Section 2 (breach-derived lists) outperforms naive dictionaries, and why Section 9 (custom-built, org-specific lists) frequently outperforms both.

PART 1: Curated & Existing Wordlists

2. Breach-Derived Corpora — rockyou.txt and Successors

The canonical starting point referenced throughout both your JtR and Hashcat guides — **real passwords from real, previously breached services**, not theoretical dictionary words:

```
rockyou.txt   → ~14 million passwords from the 2009 RockYou breach, still the most wid
ely used baseline wordlist over 15 years later, specifically because real human passwo
rd-choice behavior changes slowly
```

```
Why breach-derived beats a dictionary of real WORDS:
  Dictionary word list:  password, dragon, sunshine, ...
  Breach-derived list:   password123, Dragon2019!, sunshine95, ...
                          (captures the MANGLING patterns humans
                           actually apply, not just the base word)
```

Later, larger breach-compilation lists (commonly circulated as "COMB" - Compilation Of Many Breaches - style aggregates, running into billions of entries) trade some per-entry relevance for sheer volume — useful when targeting a very large, diverse population where you can't predict specific patterns in advance.

3. SecLists — The Curated Meta-Repository

Rather than one wordlist, SecLists is a maintained **collection of wordlists organized by use case** — passwords, usernames, web content discovery paths, fuzzing payloads, and more, pulled together from many individual sources into one consistently maintained repository:

```
SecLists/Passwords/           → various breach-derived and curated password lists, inc
luding probable-wordlists ranked by real-world frequency

SecLists/Usernames/             → common username patterns/formats

SecLists/Discovery/Web-Content/   → directory/filename bruteforcing lists (a different
discipline entirely, but same underlying "curated candidate list" concept)
```

Worth knowing this repository exists **before** reaching for a bespoke solution — a huge fraction of "I need a wordlist for X" needs are already solved by an existing, well-maintained SecLists entry.

4. Probable Wordlists — Frequency-Ranked Lists

Some curated lists are explicitly **sorted by real-world observed frequency** rather than presented as an unordered dump — meaning the most likely candidates are tried first, which matters enormously when time/compute is constrained and you may not get through the entire list:

```
Unordered rockyou.txt:  candidates tried in original breach-dump order
Frequency-ranked list:   123456, password, 12345678, qwerty, ...
                          (statistically most-common passwords FIRST)
```

For a time-boxed engagement, running a frequency-ranked list first — even a much shorter one — often yields cracks faster than running the full unordered corpus start to finish.

5. Language & Region-Specific Wordlists

Generic English-language corpora systematically underperform against populations that primarily think/type in another language — a Portuguese-speaking target population's passwords will draw on Portuguese words, regional slang, and local cultural references a rockyou.txt-style English corpus simply doesn't contain:

```
- Region-specific breach dumps (search for breaches of services popular specifically i
n the target's country/language)
- Localized dictionary wordlists (many languages have dedicated curated lists analogou
s to English rockyou.txt derivatives)
```

This is an easy factor to overlook when defaulting to "the standard list everyone uses" — worth explicitly checking target demographics before assuming an English-centric corpus is the right starting point.

6. Evaluating a Wordlist Before Committing Compute to It

Before running a large list against a slow hash (bcrypt/Argon2, per your JtR/Hashcat guides' Section 1 point about compute cost), a few quick checks are worth the couple of minutes they cost:

```
wc -l wordlist.txt                # entry count - sanity chec
k size
```

```
awk '{print length}' wordlist.txt | sort -n | uniq -c   # l
ength distribution
```

```
- Does the length distribution match the target's actual minimum password length polic
y? (a list full of 4-6 char entries is wasted effort against an 8-char-minimum policy)
- Is it deduplicated? Running redundant entries against a slow hash wastes real comput
e time for zero additional coverage
- Is the encoding consistent (UTF-8 vs Latin-1 mismatches silently corrupt non-ASCII e
ntries and waste candidates)
```

PART 2: Generating Custom Wordlists

7. Combining & Merging Existing Lists

The simplest form of customization — concatenate multiple curated sources, then deduplicate and sort:

```
cat rockyou.txt seclists-list2.txt custom-terms.txt | sort
-u > combined.txt
```

Straightforward, but worth doing deliberately rather than habitually appending every list you can find — bigger isn't automatically better once you're running against a slow hash where every extra candidate has a real time cost (your JtR/Hashcat guides' Section 7 benchmark point applies directly here).

8. CeWL — Scraping a Target's Own Website for Vocabulary

CeWL crawls a target organization's website and extracts every word it encounters, building a wordlist directly from the organization's **own vocabulary** — product names, internal jargon, executive names, department terminology — words a generic corpus would never contain but which employees plausibly incorporate into passwords:

```
cewl -d 2 -m 5 -w custom-wordlist.txt https://target-org.co
m
```

```
-d 2   → crawl depth (2 levels of links from the start page)
-m 5     → minimum word length to include
-w FILE   → output wordlist file
```

```
--with-numbers  → also extract number sequences found on the site
                    (product versions, years mentioned, etc.)
```

This is a direct, practical instance of Section 1's core lesson — a small, highly-targeted list built from the actual organization's own language frequently outperforms a much larger generic corpus, because it's modeling the *specific* population rather than the general one.

9. Building Org-Specific Lists From OSINT

Beyond website scraping, systematically compiling known facts about the target organization/individuals into raw material for mangling:

```
- Company name, product names, subsidiary/brand names
- Founded year, headquarters city, notable addresses
- Employee names (from LinkedIn, breach data, public directories)
- Pet names, sports teams, local references (from social media OSINT
  on specific high-value targets, where in-scope and authorized)
```

```
Base terms list:  Acme, AcmeCorp, Springfield, 1998, ...
Feed into mangling rules (per your JtR guide's Section 8 / Hashcat's
Section 9) to generate the full candidate space:
  Acme1998, Acme1998!, AcmeCorp98, acme_1998, ...
```

This is precisely the "known organizational password policy" scenario both cracking guides flagged as the ideal case for a targeted mask/hybrid attack — the wordlist built here is what feeds directly into that.

10. Crunch — Pure Pattern-Based Generation

Where CeWL scrapes real content, Crunch generates a wordlist purely from a **defined character set and length range** — closer in spirit to JtR/Hashcat's mask attacks (their Section 9/4 respectively) but producing a standalone file rather than generating candidates on the fly during the crack itself:

```
crunch 8 8 -t Acme%%%% -o output.txt
```

```
8 8        → min length 8, max length 8
-t Acme%%%% → template: literal "Acme" + 4 digit positions
-o FILE      → output file
```

```
Generates: Acme0000, Acme0001, Acme0002, ... Acme9999
```

Useful when you want a **persistent, reusable file** for a known structural pattern rather than relying on the cracking tool's live mask generation each run — the tradeoff being disk space, since a large pattern space produces a correspondingly large file rather than generating candidates in memory on demand.

11. Common Substitution & Mangling — Building Your Own Rule Logic

Beyond the built-in rule files referenced in your JtR/Hashcat guides, understanding the common human substitution patterns lets you build targeted custom rules rather than relying purely on generic ones:

```
Common leetspeak substitutions humans actually use:
  a → @ or 4        e → 3        i → 1 or !
  o → 0             s → $ or 5    t → 7

Common append patterns:
  + birth year / current year
  + ! or !! at the end
  + sequential digits (123, 1234)
```

Encoding these as custom rules (per your JtR guide's Section 8 rule syntax, or Hashcat's Section 9) applied to an org-specific base list (Section 9 above) is usually the highest-value custom pipeline for a real engagement — real-word relevance, times realistic mangling, times organizational specificity.

---

12. Testing / Selection Methodology

```
1. Start with a breach-derived generic corpus (Section 2) as your baseline first pass
- cheap, broad, surprisingly effective
2. Check SecLists (Section 3) before building anything custom - the need is frequently
already solved by an existing maintained list
3. Evaluate candidate lists BEFORE committing compute (Section 6), especially against
a slow hash where wasted candidates are genuinely costly
4. If the target is a specific organization, invest in Section 8's CeWL scrape and Sec
tion 9's OSINT-derived base list - this is where the highest marginal return usually l
ives on a real engagement
5. Apply mangling rules (Section 11) on top of the targeted base list rather than trea
ting rules and wordlist choice as separate decisions - they compound
6. Track which SOURCE list/generation method actually produced each crack, same report
```

```
ing principle as both cracking guides - this tells you where to invest effort on futur
e, similar engagements
```

13. Legal & Ethical Note

```
Scraping a target's website (Section 8) or compiling OSINT (Section 9) for wordlist ge
neration carries the same authorization requirement as the cracking itself, covered in
your JtR and Hashcat guides' Section 14/13 - stay within an authorized engagement's de
fined scope, and be mindful that aggressive crawling can itself constitute unwanted lo
ad
against a target's infrastructure even when the eventual cracking is authorized.
```

14. Prevention

```
Wordlist quality is the attacker-side variable this whole guide covers - the defense-s
ide countermeasures are identical to your JtR guide's Section 15 and Hashcat's Section
14, since better wordlists don't change what stops cracking, they just change how FAST
a weak policy falls:
- Screen new passwords against known-breached corpora (Section 2's own lists, ironical
ly) at signup/change time
- Favor length over complexity-driven patterns, since complexity rules are exactly wha
t Section 11's mangling logic is built to predict
- Slow/memory-hard hashing (bcrypt/Argon2) blunts the practical impact of even a perfe
ctly-targeted wordlist, since Section 1's "candidate quality" multiplier still runs in
to the same compute-cost wall covered in both cracking guides
```

15. Practical Lab Example

```
TryHackMe/HTB rooms that pair a CeWL scrape against a fictional company's site with a
subsequent JtR/Hashcat cracking step are the clearest way to see this guide's Section
1 thesis proven in a single exercise - the room's constructed passwords are usually de
liberately built from site vocabulary, making the "targeted beats generic" lesson conc
rete rather than theoretical.

Suggested practice order:
1. Run rockyou.txt against a synthetic hash set FIRST (Section 2) - establish a baseli
ne crack rate
2. CeWL-scrape a test site you control or a designated lab target (Section 8), build a
small custom list, apply best64 rules on top
3. Compare crack rate AND time against the Section 1 baseline, directly against the SA
ME hash set - the delta is the entire point of this guide made empirical
```

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-521 (Weak Password Requirements - the policy gap that makes any wordlist effective in the first place), CWE-916 (Use of Password Hash With Insufficient Computational Effort - same storage-side root cause referenced in both cracking guides)
OWASP Top 10:2025	A02:2025 Security Misconfiguration / Cryptographic Failures territory (identical mapping to JtR/Hashcat, since wordlists are the input to the same underlying attack)
CVSS	Not directly CVSS-scored - a wordlist is an attack RESOURCE, not a vulnerability; severity inherits entirely from the hash exposure and storage weakness it's used against
Cyber Kill Chain	Reconnaissance (Sections 8-9's OSINT/scraping phase, building the list) → Actions on Objectives (the resulting list feeding a cracking session per the JtR/Hashcat guides)
MITRE ATT&CK	T1110.002 (Brute Force: Password Cracking, same technique both cracking guides map to) T1593 (Search Open Websites/Domains, covering Section 8-9's OSINT-gathering phase specifically)

Quick Reference Cheat Sheet

```
CURATED LISTS (start here):
  rockyou.txt              → default breach-derived baseline (Section 2)
  SecLists repository        → maintained multi-purpose collection (Section 3)
  probable-wordlists           → frequency-ranked, most-likely-first (Section 4)
  language/region-specific lists  → match target's actual population (Section 5)

CUSTOM GENERATION (when generic isn't enough):
  cewl -d 2 -m 5 -w out.txt URL     → scrape target's own vocabulary
  crunch 8 8 -t Acme%%%% -o out.txt   → pure pattern-based generation
  cat a.txt b.txt | sort -u > c.txt     → merge + dedupe existing lists
  OSINT-compiled base terms + rules       → org-specific + mangling
                                            (highest value, most effort)

EVALUATE BEFORE RUNNING (especially against slow hashes):
  wc -l file.txt                         → size check
  awk '{print length}' file.txt | sort -n  → length distribution check

Root lesson: a wordlist is an approximation of a population's
password-choice behavior - the tighter that approximation matches
your ACTUAL target, the fewer candidates you need, which matters
enormously the moment you're paying (per your JtR/Hashcat guides'
Section 1) the full compute cost of a slow, hardened hash algorithm.
```